

```
!pip install -q spacy
!python -m spacy download en_core_web_sm

import spacy
import string
import csv
from collections import Counter

nlp = spacy.load("en_core_web_sm")

with open("4.1_4.2_input.txt", "r") as f:
    text = f.read()

# ---- Term Frequency ----
clean_text = text.lower().translate(str.maketrans('', '', string.punctuation))
words = clean_text.split()
freq = Counter(words)

with open("4.1_term_frequency.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Term", "Frequency"])
    for term, count in freq.most_common():
        writer.writerow([term, count])

print("Term Frequency saved to 4.1_term_frequency.csv\n")
print("Top 10 Most Frequent Terms:")
for term, count in freq.most_common(10):
    print(f"{term:<15}{count}")

# ---- Named Entity Recognition ----
doc = nlp(text)
print("\nNamed Entities Found:")
print(f"{'Entity':<25}{'Label':<10}{'Description'}")
for ent in doc.ents:
    print(f"{ent.text:<25}{ent.label_:<10}{spacy.explain(ent.label_)}")
```

twenty	CARDINAL	Numerals that do not fall under another type
One	CARDINAL	Numerals that do not fall under another type
Term Frequency-Inverse Document Frequency	ORG	Companies, agencies, institutions, etc.
one	CARDINAL	Numerals that do not fall under another type
one	CARDINAL	Numerals that do not fall under another type
NLP	ORG	Companies, agencies, institutions, etc.
One	CARDINAL	Numerals that do not fall under another type
two	CARDINAL	Numerals that do not fall under another type
GloVe	PERSON	People, including fictional
FastText	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
one	CARDINAL	Numerals that do not fall under another type
NLP	ORG	Companies, agencies, institutions, etc.
Entity Recognition	PERSON	People, including fictional
NER	ORG	Companies, agencies, institutions, etc.
A. P. J. Abdul Kalam	PERSON	People, including fictional
ISRO	ORG	Companies, agencies, institutions, etc.
India	GPE	Countries, cities, states
A. P. J. Abdul Kalam	PERSON	People, including fictional
ISRO	ORG	Companies, agencies, institutions, etc.
India	GPE	Countries, cities, states
NER	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
Retrieval-Augmented Generation	ORG	Companies, agencies, institutions, etc.
RAG	PERSON	People, including fictional
NLP	ORG	Companies, agencies, institutions, etc.
one	CARDINAL	Numerals that do not fall under another type
NLP	ORG	Companies, agencies, institutions, etc.
Indic	NORP	Nationalities or religious or political groups
English	LANGUAGE	Any named language
NLP	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
more than one	CARDINAL	Numerals that do not fall under another type
NLP	ORG	Companies, agencies, institutions, etc.
English	LANGUAGE	Any named language
NLP	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
F1	GPE	Countries, cities, states
BLEU	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
millions	CARDINAL	Numerals that do not fall under another type
Grammar	PERSON	People, including fictional
NLP	ORG	Companies, agencies, institutions, etc.
Chatbots	PERSON	People, including fictional
NLP	ORG	Companies, agencies, institutions, etc.
Dataset	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.
NLP	ORG	Companies, agencies, institutions, etc.

```

import csv

with open("4.1_4.2_input.txt", "r") as f:
    text = f.read()

punctuation = ' '!()-[]{};:'"\\",<>./?@$%^&*~''
lower_text = text.lower()
clean_text = ''.join(ch for ch in lower_text if ch not in punctuation)
words = clean_text.split()

freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1

sorted_freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)

with open("4.2_term_frequency.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Term", "Frequency"])
    for term, count in sorted_freq:
        writer.writerow([term, count])

print("Term Frequency saved to 4.2_term_frequency.csv\n")
print("Top 10 Most Frequent Terms:")
for term, count in sorted_freq[:10]:
    print(f"{term:<15}{count}")

```

Term Frequency saved to 4.2_term_frequency.csv

Top 10 Most Frequent Terms:

a	90
and	86
the	75
to	42
in	42
can	41
of	40
is	37
may	30
word	30

```
import math
```

```

documents = [
    "Natural language processing is a field of artificial intelligence.",
    "Natural language processing helps computers understand human language.",
    "Machine learning is an important part of artificial intelligence."

```

```

]

punctuation = ' '!()-[]{};:'"\\",<>./?@$%^&*~''

def clean_and_tokenize(text):
    text = text.lower()
    text = ''.join(ch for ch in text if ch not in punctuation)
    return text.split()

tokenized_docs = [clean_and_tokenize(doc) for doc in documents]
for i, tokens in enumerate(tokenized_docs, 1):
    print(f"Document {i} Tokens: {tokens}")
print()

def compute_tf(tokens):
    tf = {}
    for term in tokens:
        tf[term] = tf.get(term, 0) + 1
    for term in tf:
        tf[term] /= len(tokens)
    return tf

tf_docs = [compute_tf(tokens) for tokens in tokenized_docs]
vocab = sorted(set(t for tokens in tokenized_docs for t in tokens))

df = {term: sum(1 for tokens in tokenized_docs if term in tokens) for term in vocab}

N = len(documents)
idf = {term: math.log(N / df[term]) for term in vocab}

tfidf_docs = [{term: tf.get(term, 0) * idf[term] for term in vocab} for tf in tf_docs]

for i, (tf, tfidf) in enumerate(zip(tf_docs, tfidf_docs), 1):
    print(f"--- Document {i} ---")
    print(f"{'Term':<15}{'TF':<10}{'DF':<6}{'IDF':<10}{'TF-IDF'}")
    for term in vocab:
        if tf.get(term, 0) > 0:
            print(f"{term:<15}{tf.get(term,0):<10.4f}{df[term]:<6}{idf[term]:<10.4f}{tfidf[term]:.4f}")
    print()

for i, tfidf in enumerate(tfidf_docs, 1):
    top_terms = sorted(tfidf.items(), key=lambda x: x[1], reverse=True)[:10]
    print(f"Top TF-IDF Terms in Document {i}:")
    for term, score in top_terms:
        if score > 0:
            print(f"    {term:<15}{score:.4f}")

```

```
print()
```

```
language      0.1111  2    0.4055  0.0451
natural       0.1111  2    0.4055  0.0451
of            0.1111  2    0.4055  0.0451
processing    0.1111  2    0.4055  0.0451
```

```
--- Document 2 ---
```

Term	TF	DF	IDF	TF-IDF
computers	0.1250	1	1.0986	0.1373
helps	0.1250	1	1.0986	0.1373
human	0.1250	1	1.0986	0.1373
language	0.2500	2	0.4055	0.1014
natural	0.1250	2	0.4055	0.0507
processing	0.1250	2	0.4055	0.0507
understand	0.1250	1	1.0986	0.1373

```
--- Document 3 ---
```

Term	TF	DF	IDF	TF-IDF
an	0.1111	1	1.0986	0.1221
artificial	0.1111	2	0.4055	0.0451
important	0.1111	1	1.0986	0.1221
intelligence	0.1111	2	0.4055	0.0451
is	0.1111	2	0.4055	0.0451
learning	0.1111	1	1.0986	0.1221
machine	0.1111	1	1.0986	0.1221
of	0.1111	2	0.4055	0.0451
part	0.1111	1	1.0986	0.1221

```
Top TF-IDF Terms in Document 1:
```

```
a          0.1221
field      0.1221
artificial  0.0451
intelligence 0.0451
is         0.0451
language   0.0451
natural    0.0451
of         0.0451
processing 0.0451
```

```
Top TF-IDF Terms in Document 2:
```

learning	0.1221
machine	0.1221
part	0.1221
artificial	0.0451
intelligence	0.0451
is	0.0451
of	0.0451